

INF3201- Parallel Programming

Assignment 1 - MPI

31 August - September 18 2026

1 Parallel crackme challenge

A crackme is a challenge to evaluate the retro-engineering skills of programmer. This assignment evaluates your MPI parallelization skills, on a simple task. The Advanced Encryption Standard (AES) is a widely used symmetric encryption algorithm. AES is used to encrypt data using a password. An overview of the pipeline needed for AES encryption is provided on Figure 1. This assignment focuses on the AES-256 algorithm which takes as inputs an Initialization Vector (IV), a 256 bits key and the data to encrypt. To generate the 256 bits key needed for AES-256, a Key Derivation Function (KDF) is used. Here we use Password-Based Key Derivation Function 2 (PBDF2) as a KDF. PBDF2 takes a salt and a user supplied password as input to generate the 256 bits key.

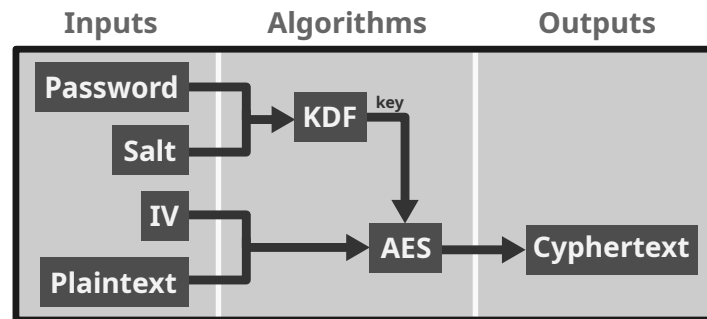


Figure 1: Overview of the AES encryption pipeline. Plaintext are the data to encrypt. Cyphertext are the encrypted data. IV is the Interrupt Vector used to initialize the AES-256 algorithm.

For simplicity reasons, the salt and IV are fixed and given. Passwords are limited to the following character set `[0-9a-zA-Z]` (digits from 0 to 10, lower and upper case letters). To help you kick-start the assignment, a couple of functions are given:

- `file_load()` load a whole file in memory.
- `file_save()` save a memory region to a file.
- `pbkdf2()` generate a 256-bits key from a password.
- `encrypt()/decrypt()` encrypt and decrypt in-memory data.
- `sha512sum()` compute the sha512 sum of in-memory data.

The sha512 checksum helps you to ensure that your decryption attempt succeeded. See `main.c` which provides the entire code to decrypt a file. An executable called `encrypt` is also given. It allows you to encrypt a file with a password using the correct IV and salt. This executable generates two files:

- `<filename>.enc` (the encrypted file)
- `<filename>.sha512` (storing the original file sha512 sum)

To keep it simple, entire files are loaded into memory for this assignment. You should make sure that enough memory is allocated for your in-memory files. In particular, the *encrypt* executable will crash if your file is too large (greater than 1MB).

2 Tasks

You first have to automate the password generation, and implement the brute-force approach in a sequential way. Your solution must search for the correct password to decrypt an AES-256 encrypted file. You can generate the various encrypted files using the *encrypt* executable. Then, you have to implement a parallel version of your solution using MPI.

Once these versions are implemented, you should propose (at least) one optimization to your parallel version. Each optimization should be backed up by evaluations, against the initial parallel version. Each version of your code should be tagged, saved and accessible for evaluation. We expect at least 3 code versions: Sequential, Parallel, Optimization_i.

To evaluate the performance of your implementations, you need to run a set of measurements. We will focus on time measurements. The sequential version will be a baseline for comparison against your parallel version(s), running on multiple nodes. Your solution should be able to scale when the *password length* and the *number of processes* vary. Try to keep the size of the encrypted files small enough to minimize its impact on the performance. Use the concepts and methodologies discussed during lectures to make your performance evaluations.

Hints:

- In this assignment, we expect you to use the *C* language.
- You will need to adapt the Makefile to compile your sequential and MPI versions

3 Requirements

3.1 Hand-in

You will work alone for this assignment. Canvas is used as a hand-in platform for the assignments. We encourage you to hand-in an archive which contains a git repository. As you work, you can and probably should commit as often as possible to keep track of your project evolution.

Remember to upload your project on Canvas before Sep 18 September 2026 23:59:59. Submission after that deadline will not be possible.

3.2 Deliverables

An archive must be delivered on Canvas containing the source code and the report.

Code – It should contain all the necessary files to compile, execute and reproduce your experiments on the cluster.

Report – Along with the source code, a report must be part of the archive. This report of about 6 to 10 pages should have the following sections:

1. **Introduction** – Introduce and explain your understanding of the project.
2. **Sequential solution** – Explain your sequential solution code, design and how to execute it.

3. **Parallel solutions** – Explain your parallelized and optimized codes, designs and how to execute them.
4. **Performance evaluation** – Present and analyze the results for your weak and strong scaling experiments.
5. **Discussion** – Arguments about positive and negative points of your solutions. Compare your results to theoretical ones. Propose some future possible improvements. If the assignment is not individual, explain how you collaborate and divided the work with your partner.
6. **Conclusion** – Summarize and conclude your work.

3.3 Grading process

Following the hand-in deadline, a first grading period of one week will be performed to evaluate your work. As a result, if you have a pass: Congratulation! You have completed the assignment. If your work does not meet the requirements, a single redo will be accorded to you. You will have one week to perform the necessary updates. Finally, a last grading period of one week will be performed to grade the redos. A fail following the second grading period is definitive, and you will not be allowed to take the exam.

To refine and guide you in your work, we provide you a list of evaluation criteria used to grade your projects. These criteria are organized into three categories namely general, code and report.

1. **General:** Ensure you handed-in the project on time; Check for cheating; Check for AI use to bypass the assignment work; Check if report and code are present; Check for fair sharing of the work with your partner.
2. **Code:** Ensure the code compiles successfully; Check for bugs/issues in the source code; Check for bugs and or crash during execution; Check if code is understandable and well commented; Check data partitioning fairness among processes/threads/cores; Ensure that computational algorithm is correct; Ensure that data reduction algorithm is correct and fair; Check if the flow of data between processes/threads/core is done only via MPI/OpenMP/CUDA API (e.g., not relying on NFS); Assess if your code executes on multiple nodes/cores; Ensure you avoid or have a smart use of oversubscribe; Check your optimized versions; Check if you tried to minimize synchronization and communications; Ensure your time gathering is accurate.
3. **Report:** Ensure you provide documentation and explanations on code and how to execute it; Check that you provide hardware details; Ensure you provide any graphs or tables to analyze your results; Check your strong scaling experiments; Check the degree of parallelism used in your strong scaling experiments is large enough; Check your weak scaling experiments; Check that your problem size and degree of parallelism large enough for your weak scaling experiments; Check that you report time, speedup and efficiency metrics; Ensure that your results are reproducible.